Homework 3 – Intraday Trading with Linear and Logistic Predictors

1 Introduction

This project implements and evaluates supervised-learning trading strategies for intraday stock trading. At each trading day t, the decision is taken at the market *open* and the profit/loss is realized at the same-day *close*. The key constraint is that all predictors must use only information available at (or before) the opening time, i.e., no leakage from same-day closing prices.

We study two strategies:

- **Strategy A (Linear regression, one-stock-per-day):** predict each stock's same-day closing price and invest 100% of capital in the single stock with maximal predicted intraday return.
- **Strategy B (Logistic regression, probability weights):** estimate the probability of a positive intraday return and allocate capital using probability-driven weights (with sparsification and risk scaling).

All results are reported on the **test period July–December 2018** with initial capital $C(0) = 10{,}000$ euros.

2 Problem Statement and Mathematical Setup

We consider $K = 10$ stocks. For each stock k and day t, we observe:

$$\mathrm{Open}_k(t), \quad \mathrm{Close}_k(t), \quad \mathrm{Volume}_k(t).$$

The realized **intraday return** (open-to-close) is:

$$R_k(t) = \frac{\mathrm{Close}_k(t) - \mathrm{Open}_k(t)}{\mathrm{Open}_k(t)}. \tag{1}$$

A portfolio is described by daily weights $w_k(t)$ satisfying long-only full-investment:

$$w_k(t) \geq 0, \qquad \sum_{k=1}^{K} w_k(t) = 1. \tag{2}$$

The portfolio's daily return and capital dynamics are:

$$r_{\mathrm{port}}(t) = \sum_{k=1}^{K} w_k(t)\, R_k(t), \qquad C(t+1) = C(t)\big(1 + r_{\mathrm{port}}(t)\big). \tag{3}$$

The goal is to design predictors and $w_k(t)$ to maximize the final capital on the evaluation period.

3 Data, Alignment, and Splits

3.1 Data Alignment

The dataset contains 10 tickers (AAPL, ADBE, AMD, AMZN, AVGO, FB, GOOGL, MSFT, QCOM, TWTR). All stocks are aligned on a common set of trading days (intersection of dates) so that each date corresponds to a consistent K-dimensional vector of prices and volumes.

3.2 Chronological Split

To prevent look-ahead bias, the split is chronological:

- **Train:** Jan 1 – May 31, 2018
- **Validation:** Jun 1 – Jun 30, 2018 (hyperparameter tuning)
- **Test:** Jul 1 – Dec 31, 2018 (final evaluation)

4 Feature Engineering (Open-Known, No Leakage)

For each stock k, we build a feature vector $x_k(t) \in \mathbb{R}^d$ using only information available at the market open of day t. To enforce this constraint, all features using close/volume are computed from days $\leq t-1$ (i.e., shifted), while open-known quantities (e.g., today's open) may be used directly.

Moreover, the selected features are commonly used in the financial literature and practice for modeling short-term price dynamics.

4.1 Python Codes

```python
def make_features(
    Open: pd.DataFrame,
    Close: pd.DataFrame,
    Vol: pd.DataFrame,
    lags: int = 3
) -> dict[str, pd.DataFrame]:
    """Feature engineering.
    All features must be known at market open on day t.
    We therefore use ONLY lagged information (shift >= 1).
    """
    feats: dict[str, pd.DataFrame] = {}

    # Same-day intraday return (used ONLY to create *lagged* predictors)
    R = (Close - Open) / Open

    for tkr in Open.columns:
        df = pd.DataFrame(index=Open.index)

        # --- Core lagged predictors ---
        # Recent intraday returns (very short memory works best for intraday)
        for k in range(1, lags + 1):
            df[f"ret_l{k}"] = R[tkr].shift(k)

        # Volume surprise (yesterday volume vs its recent average)
        df["vol_l1"] = Vol[tkr].shift(1)
        df["dvol_l1"] = Vol[tkr].shift(1) / (Vol[tkr].shift(2) + 1e-8) - 1.0
        df["vol_ratio"] = Vol[tkr].shift(1) / (Vol[tkr].shift(1).rolling(5).
            mean() + 1e-8)

        # Overnight gap: today's open vs yesterday close
        df["gap"] = (Open[tkr] - Close[tkr].shift(1)) / Close[tkr].shift(1)

        # Previous-day intraday move (mean-reversion / continuation proxy)
        df["prev_intraday"] = (Close[tkr].shift(1) - Open[tkr].shift(1)) / Open
            [tkr].shift(1)
```

```python

        # Rolling volatility of intraday returns (risk regime), strictly lagged
        df["vol_5"] = R[tkr].shift(1).rolling(5).std()

        # ---------- Experienced quant features (all open-known / lagged) ----------
        # Gap momentum
        df["gap_ma3"] = df["gap"].rolling(3).mean()
        df["gap_ma10"] = df["gap"].rolling(10).mean()

        # Overnight returns (today open vs past closes)
        df["overnight_ret1"] = (Open[tkr] / Close[tkr].shift(1) - 1.0)
        df["overnight_ret5"] = (Open[tkr] / Close[tkr].shift(5) - 1.0)

        # Intraday momentum (yesterday and rolling means)
        intr = R[tkr]
        df["intraday_1"] = intr.shift(1)
        df["intraday_3"] = intr.shift(1).rolling(3).mean()
        df["intraday_5"] = intr.shift(1).rolling(5).mean()

        # Volatility regime
        df["vol_10"] = intr.shift(1).rolling(10).std()
        df["vol_ratio_5_10"] = df["vol_5"] / (df["vol_10"] + 1e-12)

        # Abnormal volume (z-score + ratio to 20d mean), lagged
        v1 = Vol[tkr].shift(1)
        v20 = Vol[tkr].shift(1).rolling(20).mean()
        v20s = Vol[tkr].shift(1).rolling(20).std()
        df["vol_z20"] = (v1 - v20) / (v20s + 1e-12)
        df["vol_ratio20"] = v1 / (v20 + 1e-12)

        # Trend proxy from past closes (MA crossover)
        c1 = Close[tkr].shift(1)
        ma5 = c1.rolling(5).mean()
        ma10 = c1.rolling(10).mean()
        df["ma_diff_5_10"] = (ma5 - ma10) / (ma10 + 1e-12)

        df = df.replace([np.inf, -np.inf], np.nan)
        feats[tkr] = df.dropna()

    return feats
```

4.2 Feature List (d = 23)

The implemented model uses $d = 23$ features:

Group	Features
Lagged intraday returns	`ret_l1, ret_l2, ret_l3, ret_l4, ret_l5`
Lagged volume & dynamics	`vol_l1, dvol_l1, vol_ratio, vol_z20, vol_ratio20`
Overnight / gap features	`gap, gap_ma3, gap_ma10, overnight_ret1, overnight_ret5`
Lagged intraday aggregates	`prev_intraday, intraday_1, intraday_3, intraday_5`
Volatility regime	`vol_5, vol_10, vol_ratio_5_10`
Trend signal	`ma_diff_5_10`

4.3 Input Shapes

For each stock k, the supervised learning matrices have the form:

$$X_k \in \mathbb{R}^{T \times d}, \qquad d = 23, \tag{4}$$

where T is the number of usable days after applying lags and dropping missing rows. The global feature construction output shape is $(1305, 23)$ per stock (usable history across years in the dataset).

Two different targets are used:

- **Regression target:** $y_k^{\text{lin}}(t) = \text{Close}_k(t)$
- **Classification target:**

$$y_k^{\log}(t) = \mathbb{I}\{\text{Close}_k(t) > \text{Open}_k(t)\}. \tag{5}$$

5 Strategy A: Linear Regression (One-Stock-Per-Day)

5.1 What We Did (Step-by-Step)

For each stock k:

1. Construct open-known features $x_k(t)$ (Section 4).
2. Fit a linear regression model on the training period.
3. Predict same-day close $\widehat{\text{Close}}_k(t)$ on test days.
4. Convert predictions into predicted intraday return $\widehat{r}_k(t)$.
5. Each day, choose the single stock with maximum $\widehat{r}_k(t)$ and invest all capital in it.

5.2 Model and Estimation

The per-stock linear predictor is:

$$\widehat{\text{Close}}_k(t) = w_k^\top x_k(t) + b_k, \tag{6}$$

where (w_k, b_k) are fitted by least squares over the training set:

$$(w_k, b_k) = \arg\min_{w,b} \sum_{t \in \mathcal{T}_{\text{train}}} \Big(\text{Close}_k(t) - (w^\top x_k(t) + b)\Big)^2. \tag{7}$$

The predicted intraday return is computed using the observed open:

$$\widehat{r}_k(t) = \frac{\widehat{\text{Close}}_k(t) - \text{Open}_k(t)}{\text{Open}_k(t)}. \tag{8}$$

5.3 Trading Rule (Hard Argmax Allocation)

Define the selected stock:

$$k^\star(t) = \arg\max_{k \in \{1,\dots,K\}} \widehat{r}_k(t). \tag{9}$$

Then the portfolio weights are one-hot:

$$w_k(t) = \begin{cases} 1, & k = k^\star(t), \\ 0, & \text{otherwise}. \end{cases} \tag{10}$$

This implements a fully concentrated strategy, meaning that prediction errors can strongly affect capital (high variance).

5.4 Python Codes

```
X = FEATS[tkr].dropna()
y = Close[tkr].reindex(X.index)

Xtr, ytr = X.loc[:TRAIN_END], y.loc[:TRAIN_END]
Xte      = X.loc[TEST_START:TEST_END]

lr = LinearRegression().fit(Xtr, ytr)
pred_close.loc[Xte.index, tkr] = lr.predict(Xte)

pred_ret = (pred_close - Open) / Open
```

```
best = pred_ret.loc[test_idx].fillna(-1e9).idxmax(axis=1)

W = pd.DataFrame(0.0, index=test_idx, columns=TICKERS)
for dt, tkr in best.items():
    W.loc[dt, tkr] = 1.0
```

6 Strategy B: Logistic Regression (Probability Weights)

6.1 What We Did (Step-by-Step)

For each stock k:

1. Define the label $y_k(t)$ from Eq. (5).
2. Train a logistic regression model (with feature standardization) on the training set.
3. Predict probabilities $p_k(t) = \mathbb{P}(y_k(t) = 1 \mid x_k(t))$ on validation and test sets.
4. Tune portfolio hyperparameters on validation (threshold, top-K, volatility window, etc.).
5. Convert probabilities into daily portfolio weights $w_k(t)$ and backtest on test.

6.2 Probabilistic Model

Logistic regression models the conditional probability:

$$p_k(t) = \mathbb{P}(y_k(t) = 1 \mid x_k(t)) = \sigma\left(\beta_k^\top \tilde{x}_k(t)\right), \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{11}$$

where $\tilde{x}_k(t)$ denotes standardized features (zero mean, unit variance) and β_k is learned.

The negative log-likelihood minimized on training is:

$$\beta_k = \arg\min_{\beta} \sum_{t \in \mathcal{T}_{\text{train}}} \Big[-y_k(t)\log p_k(t) - (1-y_k(t))\log(1-p_k(t))\Big] + \lambda\|\beta\|_2^2. \tag{12}$$

6.3 From Probabilities to Weights

We map probabilities to a **buying strength** via a threshold θ:

$$s_k(t) = \max\big(p_k(t) - \theta,\, 0\big). \tag{13}$$

Sparsification (Top-K_{top}). To avoid spreading capital too thinly, we keep only the K_{top} largest strengths and set the rest to zero:

$$s_k(t) \leftarrow 0 \quad \text{if } k \notin \text{Top-}K_{\text{top}}\{s_1(t), \ldots, s_K(t)\}.$$

Risk scaling using past volatility. Let $\widehat{\sigma}_k(t)$ be a rolling volatility estimate computed only from *past* returns:

$$\widehat{\sigma}_k(t) = \text{std}\big(R_k(t-1), R_k(t-2), \ldots, R_k(t-W)\big). \tag{14}$$

A volatility cap $\sigma_{\max}$ prevents extreme leverage-like effects. The risk-adjusted strength is:

$$\tilde{s}_k(t) = \frac{s_k(t)}{\min(\widehat{\sigma}_k(t), \sigma_{\max})^{\gamma} + \varepsilon}, \tag{15}$$

where γ (called `power` in code) controls how aggressively we penalize volatility.

Normalization. Finally, we produce long-only fully invested weights:

$$w_k(t) = \frac{\tilde{s}_k(t)}{\sum_{j=1}^{K} \tilde{s}_j(t)}. \tag{16}$$

6.4 Python Codes

```python
y = (Close[tkr] > Open[tkr]).astype(int).reindex(X.index)

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("logit", LogisticRegression(C=0.5, class_weight="balanced", max_iter=2000)
        )
]).fit(Xtr, ytr)

prob.loc[Xte.index, tkr] = clf.predict_proba(Xte)[:, 1]
```

```python
strength = (prob_day - buy_th).clip(lower=0.0)          # Eq. threshold

strength = strength.where(strength.rank(ascending=False) <= top_k, 0.0)

vol = R.shift(1).rolling(vol_win).std().loc[day]        # past-only risk
w = strength / (vol + 1e-8)                             # inverse-vol scaling
w = w / w.sum()                                         # normalize to 1
```

6.5 Validation Tuning (Selected Hyperparameters)

We performed a grid search on the validation period and selected the following hyperparameters:

$$\theta = 0.55, \quad K_{\text{top}} = 1, \quad W = 3, \quad \gamma = 1.0, \quad \sigma_{\max} = 0.02.$$

This means the final logistic strategy is a *probability-filtered, volatility-scaled, sparse allocation* that typically picks the strongest single candidate each day, but with probability and risk discipline.

7 Backtesting Details

7.1 Continuous Weights

Given $w_k(t)$, daily portfolio return and capital follow Eq. (3).

```
port_ret = (W * R).sum(axis=1)
C = C0 * (1 + port_ret).cumprod()
```

7.2 Integer-Share Simulation

For realism, weights are also converted into integer shares at the open:

$$N_k(t) = \left\lfloor \frac{w_k(t)\, C(t)}{\mathrm{Open}_k(t)} \right\rfloor, \tag{17}$$

leaving leftover cash uninvested. The close-of-day capital is then computed using $\mathrm{Close}_k(t)$ plus leftover cash. This tests whether discretization changes conclusions.

```
alloc  = W.loc[day] * cap
shares = np.floor(alloc / Open.loc[day]).astype(int)
cap = cap - (shares * Open.loc[day]).sum() \
      + (shares * Close.loc[day]).sum()
```

8 Results

8.1 Capital Evolution (Test: Jul–Dec 2018)

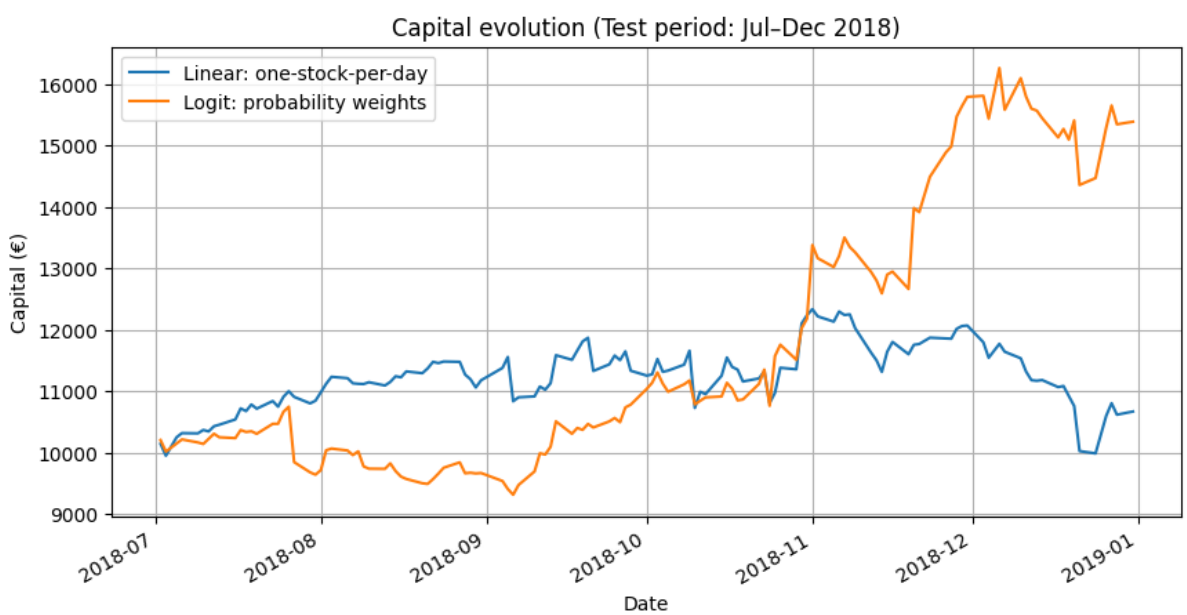

Figure 1: Capital evolution on test period (Jul–Dec 2018): Linear one-stock-per-day vs Logistic probability weights.

8.2 Continuous vs Integer Shares (Logistic Strategy)

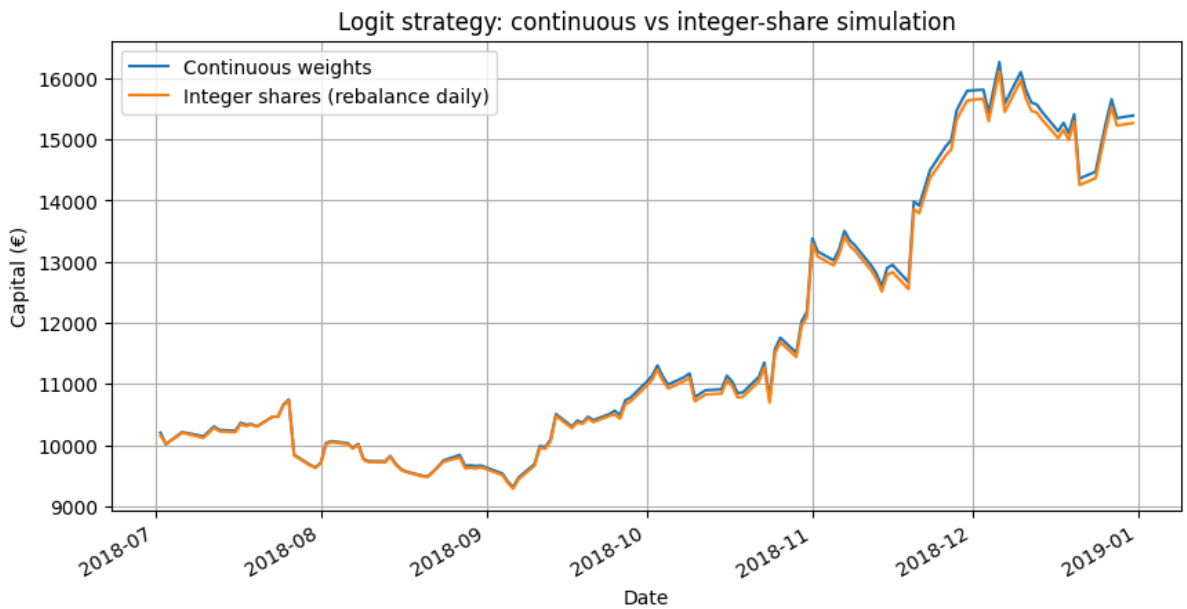

Figure 2: Logistic strategy: continuous weights vs integer-share simulation (daily rebalance).

8.3 Final Numerical Results

Table 1: Final test performance (starting capital $C(0) = 10{,}000$ euros).

Strategy	**Final Capital**	**Profit**	**Notes**
Linear regression (one-stock-per-day)	10666.67	6.67%	Hard argmax on predicted return
Logistic regression (probability weights, cont.)	15387.85	**53.88%**	Tuned on validation (threshold + top-K + volatility scaling)
Logistic regression (probability weights, integer)	15264.95	52.65%	Integer shares with leftover cash carried forward

8.4 Additional Plots

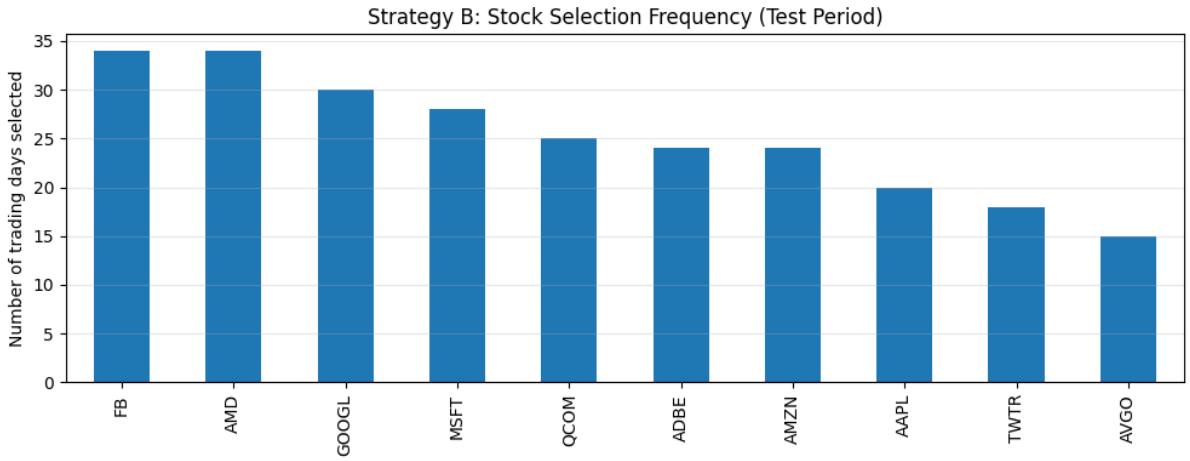

Figure 3: Logistic strategy: stock selection frequency over test time.

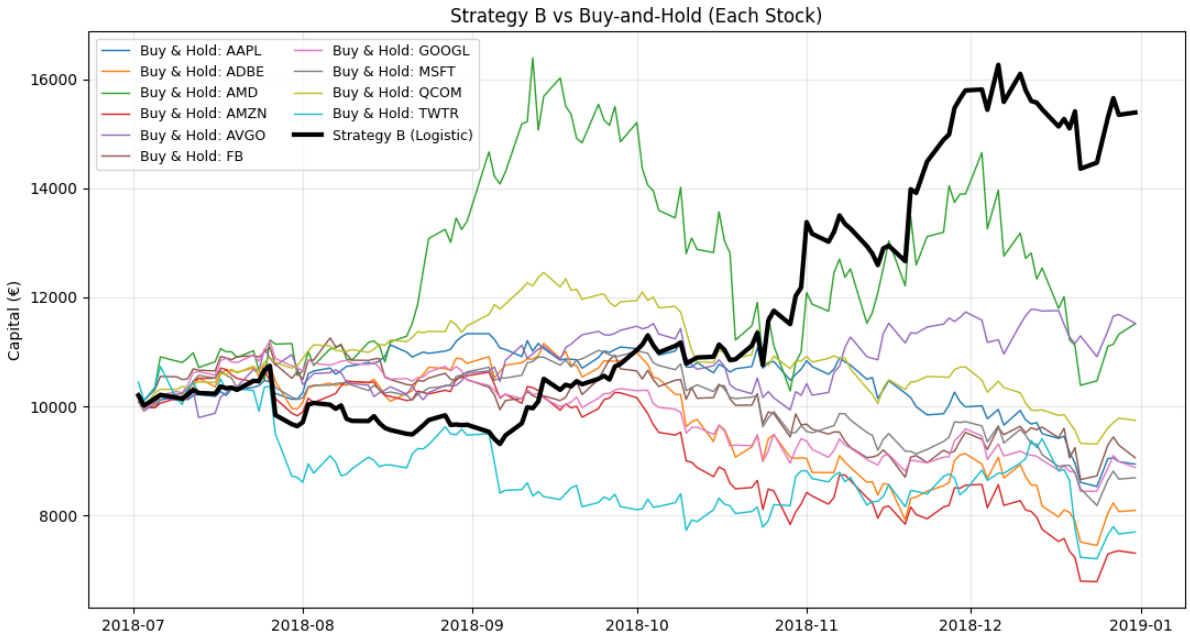

Figure 4: Logistic strategy: capital for each stock and strategy B during the test time.

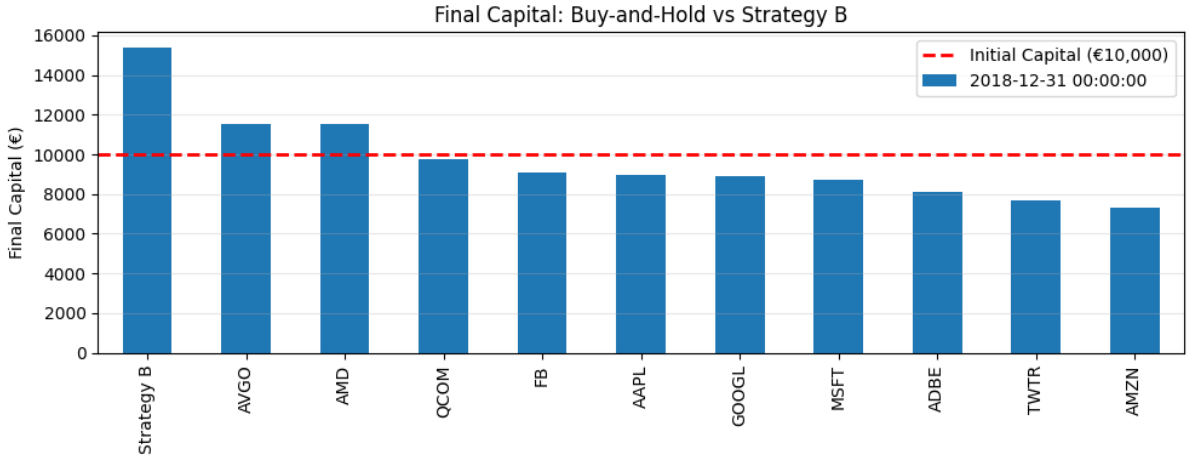

Figure 5: Logistic strategy: capital for each stock and strategy B at the end of the test time.

8.5 Interpretation

The linear strategy uses a fully concentrated allocation (Eq. (10)). This maximizes sensitivity to daily prediction errors: a single wrong pick immediately affects the entire portfolio.

The logistic strategy combines:

- probabilistic filtering via thresholding (Eq. (13)),
- sparsification (top-K_{top}),
- risk control via rolling volatility scaling (Eq. (15)),

which yields substantially higher test capital and remains stable under integer-share constraints (Fig. 2).

9 Conclusion

We built an intraday trading pipeline under strict information constraints:

- engineered 23 open-known, lagged features per stock,
- trained per-stock linear regression and logistic regression models,
- designed two trading rules: hard argmax (linear) and probability-weighted allocation (logistic),
- validated the logistic allocation hyperparameters on June 2018 and evaluated on Jul–Dec 2018,
- verified realism via integer-share simulation.

On the test period, logistic probability-weighted allocation achieved **15387.85** euros (**+53.88%**) versus **10666.67** euros (**+6.67%**) for the linear one-stock-per-day strategy, with very similar performance under integer shares.

References

[1] U. Spagnolini, *Statistical Signal Processing in Engineering*, John Wiley & Sons, Hoboken, NJ, 2018.

[2] R. S. Tsay, *Analysis of Financial Time Series*, 3rd ed., Wiley, 2010.

[3] J. Y. Campbell, A. W. Lo, and A. C. MacKinlay, *The Econometrics of Financial Markets*, Princeton University Press, 1997.

[4] E. F. Fama and K. R. French, "The Cross-Section of Expected Stock Returns," *Journal of Finance*, vol. 47, no. 2, pp. 427–465, 1992.

[5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer, 2009.

[6] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*, 5th ed., Wiley, 2012.

[7] NASDAQ, Inc., "NASDAQ Market Activities." https://www.nasdaq.com, accessed: 2025-02-09.